Day 12 Evidence Workbooklet

While Loops, Update Order, and Termination Evidence

Engineering practice to condition check to update order to termination to safety note

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/50		Mastery check	secure / developing / needs support	

Concrete engineering situation

Some checkout processes do not know in advance how many passes are needed. A kit may keep charging until it reaches a threshold, or a user may keep trying until valid input arrives. Day 12 focuses on while-loop termination evidence.

Engineering task	Decide whether a repeated checkout process will stop safely and produce the intended final state.
Computational move	Use a while condition, update statement, and termination check to repeat while a condition is true.
Python focus	while, condition checked before each pass, update order, sentinel value, infinite-loop risk, and final state.
Evidence to keep	condition value before pass, state before update, update amount, state after update, stop condition, and final printed value.

Day 12 working rules

1. Python checks the while condition before each pass.
2. The loop body must change something that can make the condition false.
3. Trace the state before and after the update.
4. A missing or wrong update can create an infinite loop.
5. The first false condition ends the loop.

Evidence map **Manage your evidence** blueprint

Part	Section	Evidence focus
1	Read a while loop as condition plus update	recognize, predict
2	Trace update order until the loop stops	trace, termination
3	Find an infinite-loop risk	debug, safety
4	Use a sentinel-style stopping condition	sentinel, trace
5	Repair a while-loop boundary	debug, boundary
6	Transfer and support route	transfer, reflect

1 Read a while loop as condition plus update

recognize predict

The kit charges in steps of 10 until the charge is no longer below 80.

```
charge_percent = 50
minutes = 0

while charge_percent < 80:
    charge_percent = charge_percent + 10
    minutes = minutes + 5

print(charge_percent, minutes)
```

Pts.	Prompt	My evidence / response
1	What condition is checked before each pass?	
1	Which statement changes charge_percent?	
1	Which statement changes minutes?	
1	What final values are printed?	

2 Trace update order until the loop stops

trace termination

Complete the trace table. The condition is checked before each row begins.

```
charge_percent = 50
minutes = 0
while charge_percent < 80:
    charge_percent = charge_percent + 10
    minutes = minutes + 5
```

Pts.	Prompt	My evidence / response
2	Before pass 1: condition value, charge, minutes. After pass 1: charge, minutes.	
2	Before pass 2: condition value, charge, minutes. After pass 2: charge, minutes.	
2	Before pass 3: condition value, charge, minutes. After pass 3: charge, minutes.	
2	What condition value is checked after charge reaches 80?	
2	Why does the loop stop at 80 instead of continuing to 90?	

3 Find an infinite-loop risk

debug safety

A loop can be syntactically valid but unsafe. Look for the missing state change.

```
charge_percent = 50

while charge_percent < 80:
    print("charging")
```

Pts.	Prompt	My evidence / response
2	What value stays unchanged?	
2	Why does the condition stay true?	
2	What line should be added to move the loop toward termination?	
2	Where should that line go?	
2	Name the evidence that would prove the repaired loop stops.	

Common mistake to avoid

A while loop needs an update that can eventually make the condition false.

4 Use a sentinel-style stopping condition

sentinel trace

A sentinel is a value that tells the loop to stop. This page traces a short manual-input pattern.

```
entry = "retry"
attempts = 0

while entry != "done":
    attempts = attempts + 1
    entry = "done"

print(attempts)
```

Pts.	Prompt	My evidence / response
2	What is the starting value of entry?	
2	Why does the loop body run the first time?	
2	Which line changes the sentinel value?	
2	How many attempts print?	
2	What would happen if entry never changed to done?	

5 Repair a while-loop boundary

debug boundary

The intended rule is to keep charging until charge is at least 80. Evaluate the condition.

```
charge_percent = 70
while charge_percent <= 80:
    charge_percent = charge_percent + 10
print(charge_percent)
```

Pts.	Prompt	My evidence / response
2	Trace the values printed by the loop state: 70 to what final value?	
2	Why does <= overshoot the intended stop boundary?	
2	Write the corrected while condition.	
2	Name a boundary test for 80.	
2	What final value should print after the correction?	

6 Transfer and support route

transfer reflect

Close Day 12 by naming the difference between known-repeat for loops and condition-controlled while loops.

Pts.	Prompt	My evidence / response
4	Write a short note explaining how a while loop decides whether to run again.	
2	Circle your support route: condition check / update order / termination / sentinel / boundary. Name one next action.	

Support route

Day 13 keeps the loop trace but adds loop-control decisions: sometimes a pass should stop early or skip the rest of the body.